

```
CREATE DATABASE college_db;
```

```
USE college_db;
```

Step 3: Create department Table

```
CREATE TABLE department (
```

```
deptno INT,
```

```
deptname VARCHAR(30),
```

```
location VARCHAR(30)
```

```
);
```

Step 4: Add designation Column

sql

Copy code

```
ALTER TABLE department
```

```
ADD designation VARCHAR(30);
```

Step 5: Insert Values into the Table

sql

```
INSERT INTO department VALUES (1, 'Computer Science', 'Pune', 'HOD');
```

```
INSERT INTO department VALUES (2, 'IT', 'Mumbai', 'Manager');
```

```
INSERT INTO department VALUES (3, 'Electronics', 'Nashik', 'Supervisor');
```

```
INSERT INTO department VALUES (9, 'Mechanical', 'Nagpur', 'Engineer');
```

Step 6: List Records Grouped by deptno

sql

```
SELECT deptno, COUNT(*) AS total
```

```
FROM department
```

```
GROUP BY deptno;
```

```
UPDATE department
```

```
SET location = 'Pune',
```

```
    designation = 'Senior Engineer'
```

```
WHERE deptno = 9;
```

Step 8: Delete Column Data (Example)

```
UPDATE department
```

```
SET designation = NULL;
```

Practical no 20

PL/SQL code to retrieve the employee name, join date, and designation from employee database of an employee whose number is input by the user.

Create EMPLOYEE Table

```
CREATE TABLE EMPLOYEE (  
    EMP_NO    NUMBER PRIMARY KEY,  
    EMP_NAME  VARCHAR2(50),  
    JOIN_DATE DATE,  
    DESIGNATION VARCHAR2(50)  
);
```

2 Insert Sample Records

```
INSERT INTO EMPLOYEE VALUES (101, 'Amit Sharma',  
    TO_DATE('15-01-2021','DD-MM-YYYY'), 'Manager');  
INSERT INTO EMPLOYEE VALUES (102, 'Neha Verma',  
    TO_DATE('20-03-2022','DD-MM-YYYY'), 'Developer');  
INSERT INTO EMPLOYEE VALUES (103, 'Rahul Patil',  
    TO_DATE('10-07-2020','DD-MM-YYYY'), 'Analyst');
```

```
COMMIT;
```

3 PL/SQL Program (User Input Employee Number)


```

DECLARE
    v_emp_no      EMPLOYEE.EMP_NO%TYPE;
    v_emp_name    EMPLOYEE.EMP_NAME%TYPE;
    v_join_date   EMPLOYEE.JOIN_DATE%TYPE;
    v_designation EMPLOYEE.DESIGNATION%TYPE;
BEGIN
    -- Accept employee number from user
    v_emp_no := &emp_no;

    -- Retrieve employee details
    SELECT EMP_NAME, JOIN_DATE, DESIGNATION
    INTO v_emp_name, v_join_date, v_designation
    FROM EMPLOYEE
    WHERE EMP_NO = v_emp_no;

    -- Display output
    DBMS_OUTPUT.PUT_LINE('Employee Name : ' || v_emp_name);
    DBMS_OUTPUT.PUT_LINE('Join Date    : ' || v_join_date);
    DBMS_OUTPUT.PUT_LINE('Designation  : ' || v_designation);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found.');
```

END;

/

4 Output

```

Enter value for emp_no: 102
Employee Name : Neha Verma
Join Date    : 20-MAR-22
Designation  : Developer
```


Practical No 21

1 Create EMPLOYEE Table (with Salary)

```
CREATE TABLE EMPLOYEE (  
    EMP_NO    NUMBER PRIMARY KEY,  
    EMP_NAME  VARCHAR2(50),  
    SALARY    NUMBER(10,2)  
);
```

2 Insert Records

```
INSERT INTO EMPLOYEE VALUES (101, 'Amit Sharma', 25000);  
INSERT INTO EMPLOYEE VALUES (102, 'Neha Verma', 18000);  
INSERT INTO EMPLOYEE VALUES (103, 'Rahul Patil', 22000);  
INSERT INTO EMPLOYEE VALUES (104, 'Sneha Kulkarni', 15000);  
INSERT INTO EMPLOYEE VALUES (105, 'Karan Mehta', 30000);
```

```
COMMIT;
```

3 PL/SQL Program

(Update salary of employees earning less than average salary using CURSOR)

Assumption: Salary will be increased by 10%

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
-- Cursor to select employees with salary less than average
```

```
CURSOR emp_cursor IS
```

```
    SELECT EMP_NO, SALARY
```

```
    FROM EMPLOYEE
```

```
    WHERE SALARY < (SELECT AVG(SALARY) FROM EMPLOYEE);
```



```

v_emp_no EMPLOYEE.EMP_NO%TYPE;
v_salary EMPLOYEE.SALARY%TYPE;

BEGIN
  OPEN emp_cursor;

  LOOP
    FETCH emp_cursor INTO v_emp_no, v_salary;
    EXIT WHEN emp_cursor%NOTFOUND;

    -- Update salary by 10%
    UPDATE EMPLOYEE
    SET SALARY = SALARY + (SALARY * 0.10)
    WHERE EMP_NO = v_emp_no;

    DBMS_OUTPUT.PUT_LINE(
      'Salary updated for Employee No: ' || v_emp_no
    );
  END LOOP;

  CLOSE emp_cursor;
  COMMIT;

END;
/

```

4 Check Updated Table

```
SELECT * FROM EMPLOYEE;
```

5 Explanation (Exam-Friendly)

◆ Cursor Declaration

```
CURSOR emp_cursor IS
SELECT EMP_NO, SALARY
```

FROM EMPLOYEE
WHERE SALARY < (SELECT AVG(SALARY) FROM EMPLOYEE);

- Cursor selects employees whose salary is less than the average salary.

◆ Cursor Operations

- OPEN emp_cursor → Opens the cursor.
- FETCH → Retrieves one record at a time.
- EXIT WHEN emp_cursor%NOTFOUND → Stops loop when all records are processed.
- CLOSE emp_cursor → Closes the cursor.

◆ Update Logic

SET SALARY = SALARY + (SALARY * 0.10)

- Increases salary by 10% for each selected employee.

◆ COMMIT

- Saves updated salary values permanently in the database.

Demonstrate the use of Group by and Order by clause

Step 1: Create Table

```
CREATE TABLE employee (  
    emp_id NUMBER,  
    emp_name VARCHAR2(30),  
    department VARCHAR2(20),  
    salary NUMBER  
);
```

Step 2: Insert Data

```
INSERT INTO employee VALUES (101, 'Amit', 'IT', 40000);  
INSERT INTO employee VALUES (102, 'Neha', 'HR', 30000);  
INSERT INTO employee VALUES (103, 'Ravi', 'IT', 45000);  
INSERT INTO employee VALUES (104, 'Priya', 'HR', 32000);  
INSERT INTO employee VALUES (105, 'Kunal', 'Sales', 28000);  
INSERT INTO employee VALUES (106, 'Anita', 'Sales', 30000);
```

◆ Use of GROUP BY Clause

Example 1: Total salary department-wise

```
SELECT department, SUM(salary) AS total_salary  
FROM employee  
GROUP BY department;
```

Output (Conceptual)

Department Total_Salary

IT	85000
HR	62000
Sales	58000

Explanation

- GROUP BY department groups records department-wise
- SUM(salary) calculates total salary for each group

◆ Use of GROUP BY with COUNT

Example 2: Number of employees in each department

```
SELECT department, COUNT(emp_id) AS total_employees  
FROM employee  
GROUP BY department;
```

◆ Use of ORDER BY Clause

Example 3: Display employees in ascending order of salary

```
SELECT emp_name, department, salary  
FROM employee  
ORDER BY salary ASC;
```

 ASC → Ascending order (default)

Example 4: Display employees in descending order of salary

```
SELECT emp_name, department, salary  
FROM employee  
ORDER BY salary DESC;
```

 DESC → Descending order

◆ GROUP BY + ORDER BY Together (Very Important for Exams)

Example 5: Department-wise total salary ordered by highest salary

```
SELECT department, SUM(salary) AS total_salary
```

```
FROM employee  
GROUP BY departmnt  
ORDER BY total_salary DESC;
```

Explanation

- GROUP BY groups records
- ORDER BY sorts the grouped result

Key Points (For Viva)

- GROUP BY is used with **aggregate functions** (SUM, COUNT, AVG, MAX, MIN)
- Columns in SELECT must appear in GROUP BY (except aggregate functions)
- ORDER BY is always written **at the end**
- GROUP BY comes **before** ORDER BY

Conclusion

This practical demonstrates how **GROUP BY** is used to group data and how **ORDER BY** is used to sort the result for better analysis.

Practical no 19

Write PL/SQL code to display Employee details using Explicit Cursors

Step 1: Table (Assumed)

```
CREATE TABLE employee (  
  emp_id NUMBER,  
  emp_name VARCHAR2(20)  
);
```

Insert Data

```
INSERT INTO employee VALUES (1, 'Amit');  
INSERT INTO employee VALUES (2, 'Neha');  
INSERT INTO employee VALUES (3, 'Ravi');  
COMMIT;
```

Step 2: PL/SQL Explicit Cursor Program (Very Easy)

```
SET SERVEROUTPUT ON;  
  
DECLARE  
  CURSOR emp_cur IS  
    SELECT emp_id, emp_name FROM employee;  
  
  v_id employee.emp_id%TYPE;  
  v_name employee.emp_name%TYPE;  
  
BEGIN  
  OPEN emp_cur;  
  
  LOOP  
    FETCH emp_cur INTO v_id, v_name;  
    EXIT WHEN emp_cur%NOTFOUND;  
  
    DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_name);  
  END LOOP;  
  
  CLOSE emp_cur;  
END;  
/
```

Output

```
1 Amit  
2 Neha  
3 Ravi
```

Simple Explanation (Line by Line)

- CURSOR emp_cur IS → declares explicit cursor
- OPEN emp_cur → runs SELECT query
- FETCH → gets one row at a time
- EXIT WHEN emp_cur%NOTFOUND → stops when rows end

- CLOSE emp_cur → frees memory

An **explicit cursor** is declared by the programmer and is used to process **multiple rows** returned by a SELECT statement in PL/SQL.

Cursor:-

What is a Cursor?

A **cursor** is a **pointer** that is used to fetch and process **one row at a time** from the result set of a SQL query in PL/SQL.

Types of Cursors

1 Implicit Cursor

- Automatically created by Oracle
- Used for INSERT, UPDATE, DELETE, SELECT INTO
- Example:

```
BEGIN
  UPDATE employee SET salary = salary + 1000;
  DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows updated');
END;
/
```

2 Explicit Cursor

- Declared and controlled by the programmer
- Used when a query returns **multiple rows**